

项目申请书

基于 Agent 实现自然语言驱动的数据看板生成与发布

项目编号	26f420021	申请人	郭万宏
组织 / 项目	DB-GPT / 基础档	周期	中选后 2 个月
学校	天津工业大学	专业年级	软件工程专业大三
GitHub ID	jcsdxhe	GitHub 主页	github.com/jcsdxhe
邮箱	junchengshu@163.com	联系电话	16622156734
预计投入	每周约 30 小时	申请时间	2026 年 8 月

核心主张

本项目不是再生成一张图，而是把自然语言分析目标稳定地转化为可编辑、可刷新、可保存、可发布和可分享的 Dashboard 工程对象。

公开交互原型

[点击访问 DB-GPT Dashboard Showcase v3 编辑工作台](#)

静态交互验证 · 不调用真实模型 · 不访问生产数据库

技术方案 · 模块拆解 · 测试策略 · 六周交付计划

一、申请摘要

我对本项目的理解是：DB-GPT 已经能够完成自然语言问答、Text2SQL、Python 分析和图表/报告生成，但一次分析结果还没有被稳定地沉淀成一个可持续使用的 Dashboard 工程对象。真正缺少的不是“再画一张图”，而是一条把**业务目标、指标、维度、筛选器、组件查询、布局、编辑状态、任务上下文和发布版本**串联起来的闭环。

本项目计划在 DB-GPT 当前数据助理与 ReAct Agent 流程上，引入统一、版本化的 **DashboardSchemaV1**。Agent 先规划业务主题、指标、维度、筛选器和组件，再逐组件生成参数化 SQL；服务端负责 Schema 校验、SQL 只读安全检查、真实试运行、字段映射校验和局部失败保留；同一套前端编辑器既可以嵌入当前任务，也可以从资产库独立打开；发布时保存不可变 Schema 与数据快照，匿名分享页只读取快照，不直接访问生产数据库。

我认为最自然的用户路径应当是：用户在数据助理中提出自然语言需求，Agent 完成规划与查询后，Dashboard 自动出现在当前任务右侧；用户可以继续与数据助理对话，也可以手工编辑、刷新、保存、发布和分享。左侧“数据看板”只作为已保存资产的管理入口，而不是要求用户离开任务后再寻找刚生成的结果。

申请阶段可行性验证

为降低方案停留在概念层的风险，我在与个人学习仓库隔离的私有工作区中，对官方 **main** 基线进行了技术预研，并制作了公开交互原型。当前证据包括：

- 完整预研原型：Dashboard Schema、草稿/修订/发布 API、安全查询执行、Schema 驱动编辑器、Walmart 与 Apple 两个案例；
- 后端可复现实验：103 项通过，本地未提供测试库时 2 个真实 MySQL 用例按条件跳过，MySQL 8.4.10 GitHub Actions 已通过；
- 前端预研：9 个 Vitest 文件、15 项测试通过，Dashboard 专用 TypeScript 0 错误；
- 公开 Showcase v3：17 个测试文件、54 项测试通过，6 条真实浏览器流程通过，静态构建 9 个页面；
- 公开演示版明确为静态交互原型，不调用真实模型、不执行生产 SQL，也不把演示数据包装成真实经营结论。

现有原型不会提前向 DB-GPT 官方仓库提交 PR，也不等同于开源之夏正式成果。若中选，我会先与导师重新确认范围，再按社区规范整理模块、测试、文档和提交历史，独立完成正式实现。

二、我对项目问题的理解

2.1 当前能力与关键缺口

当前数据助理链路可以概括为：

自然语言问题 → Agent / LLM → SQL 或分析脚本 → 图表 / HTML 报告

这条链路能回答一次问题，却还不能自然支持“保存后继续编辑、调整筛选器后刷新、检查单组件 SQL、改变布局、发布稳定版本并分享给团队”等 Dashboard 使用方式。要补齐这些能力，需要解决以下九个工程问题：

- 1 Agent、后端和前端缺少统一、可版本化的数据合同；
- 2 模型输出不稳定，不能直接作为数据库执行或前端渲染的最终指令；
- 3 指标口径、维度、筛选器、SQL 参数、输出字段与图表编码必须一致；
- 4 一个组件失败时，不能丢失其他成功组件和整个草稿；
- 5 任务页与看板资产库不能维护两套 Schema、编辑器或数据；
- 6 刷新浏览器、切换历史任务后，仍应找到并继续编辑原 Dashboard；
- 7 草稿保存需要避免并发编辑时静默覆盖；
- 8 发布链接不能成为绕过权限、匿名执行生产 SQL 的入口；
- 9 新模块需要兼容 DB-GPT 现有数据源、Agent 事件流和旧图表数据。

2.2 核心目标

- 自然语言分析目标
- 规划指标、维度、筛选器与组件
 - 生成并校验 Dashboard Schema
 - 逐组件生成、校验和试运行参数化 SQL
 - 在当前任务内打开可编辑草稿
 - 编辑、保存、刷新和历史恢复
 - 发布不可变快照
 - 生成可筛选的只读分享页

2.3 项目边界

两个月内优先保证官方要求中的主链路完整，不把企业级平台扩张当作阻塞项。v1 不以完整多租户 RBAC、字符级实时协作、跨库分布式查询、定时告警、匿名实时查库和复杂手机端编辑为验收门槛。现有权限/审计原型只作为安全边界与后续扩展证明，不挤占 Schema、Agent 规划、查询绑定、编辑、刷新、发布和两个案例的开发时间。

三、完整用户流程与总体架构

3.1 用户流程

- 1 用户在数据助理中选择数据源，用自然语言描述分析目标；
- 2 Agent 读取相关表结构、字段类型和说明，先生成不含 SQL 的看板计划；
- 3 用户确认或修改“指标-维度-筛选器-组件”计划；
- 4 系统逐组件生成参数化 SQL，并完成安全校验、试运行和字段映射校验；
- 5 成功组件与失败组件共同组成可恢复草稿，失败组件保留错误与修复入口；
- 6 dashboard.created 事件触发任务右侧嵌入式编辑器，不跳离当前任务；
- 7 默认状态下，用户点击任一组件右上角的编辑图标即可打开该组件的手工设置；开启“框选批注”后，也可选中组件/数据点并用自然语言提交修改意见；
- 8 第一次保存后，草稿成为正式资产并出现在“数据看板”资产库；
- 9 发布生成不可变快照与分享链接；分享查看者可在冻结数据切片内改变筛选条件，但不能修改草稿或执行数据库 SQL。

3.2 分层架构



3.3 设计原则

- **Schema 是合同**：模型输出可变，持久化与渲染必须稳定；
- **任务页是主入口**：生成结果与上下文不割裂；
- **编辑器只有一套核心**：嵌入式和独立模式共享渲染、筛选、保存与错误处理；

- **模型不能绕过校验**：无论首次生成还是自然语言修改，都必须重新校验；
- **局部失败可恢复**：错误组件可修复，成功组件不丢失；
- **发布与草稿分离**：分享的稳定性不受后续编辑影响；
- **证据可复现**：每项“已通过”对应固定环境、命令与测试报告。

四、Dashboard Schema v1 设计

Schema 同时提供三种一致表示：Python/Pydantic 作为后端权威模型，JSON Schema 2020-12 作为跨语言合同与样例验证，TypeScript 类型用于前端状态和编辑器约束。

模块	主要内容	关键校验
schema_version	固定为 1.0	不支持的主版本拒绝加载，升级走显式迁移
dashboard	ID、标题、说明、数据源、主题、状态、时间	数据源必须存在且调用者可使用
metric_context	粒度、时间范围、数据新鲜度、来源与口径	单位、粒度和数据来源必须可解释
filters	日期范围、单选、多选、默认值	参数名唯一，值类型与 SQL 参数一致
widgets	类型、标题、查询绑定、字段映射、样式、错误状态	查询输出必须满足图表编码
layouts	12 列桌面布局、移动端堆叠	位置不重叠，宽高满足组件最小值
metadata	会话 ID、来源回合、Agent 生成信息、兼容信息	能从资产返回原任务与生成回合

4.1 Widget 查询绑定

每个组件不仅保存图表类型，还保存：数据源 ID、参数化 SQL、筛选器到 SQL 参数的映射、输出字段及类型、图表编码、查询超时、最大行数、最近执行状态、数据粒度和刷新时间。这样编辑器才能回答“这张图的数据来自哪里、筛选器怎样影响查询、为什么失败、该如何修复”。

首批正式组件为 `kpi`、`line`、`bar`、`pie`、`table`；首批全局筛选器为 `date_range`、`select`、`multi_select`。公开原型中展示的面积图、堆叠图、散点图、热力图等扩展视觉类型属于可选增强，不改变核心 Schema 和两个月主任务优先级。

4.2 任务关联与资产生命周期

核心关联字段包括 `origin`、`asset_state`、`conversation_id`、`source_turn_id`、`saved_at`。Agent 创建时为 `origin=task`、`asset_state=generated`；第一次明确保存后转为 `saved` 并进入资产库；发布时固化新的不可变修订，但可编辑草稿仍独立存在。历史任务返回精简 `dashboard_refs`，使用户刷新或重新打开会话后仍能继续编辑。

五、Agent 规划、生成与修改闭环

5.1 两阶段生成

我不会让模型一次输出包含所有 SQL 的大段 JSON，而采用“先业务计划、后逐组件查询”的两阶段方式：

- 1 读取当前数据源相关表结构、字段类型与说明；
- 2 Agent 生成不含 SQL 的 `DashboardPlan`；
- 3 服务端标准化并校验指标、维度、筛选器和推荐组件；
- 4 逐组件生成参数化 SQL；
- 5 每个 SQL 独立通过安全检查与真实试运行；
- 6 校验查询输出字段与图表编码；
- 7 成功组件正常保存，失败组件保留错误与可修复状态；
- 8 通过稳定 SSE 事件返回进度和 Dashboard ID；
- 9 前端在当前任务中自动打开嵌入式编辑器。

稳定事件包括：`dashboard.plan.started`、`dashboard.plan.completed`、`dashboard.widget.started`、`dashboard.widget.validated`、`dashboard.widget.failed`、`dashboard.created`。

5.2 自然语言修改已有看板

公开原型验证了“框选-批注-提交修改意图”的交互方式。正式实现中，前端提交的不是模糊截图，而是结构化上下文：Dashboard ID、当前修订、筛选条件、目标组件/数据点、Schema 片段和用户评论。Agent 生成的是受限 Patch/ChangeSet；服务端应用前重新执行 Schema 与查询校验，成功后进入撤销/重做和修订记录。模型不能直接改生产数据，也不能绕过 SQL 安全层。

5.3 为什么这种方式更可靠

- 业务规划与 SQL 错误分开定位；
- 单个 SQL 失败不会浪费其他成功组件；
- 同一组件可被局部修复，不必重新生成整张看板；
- 前端无需解析模型自然语言来猜测进度；
- 变更前后有结构化差异，便于审计、撤销和测试。

六、任务主链可行性原型

下图展示申请阶段公开原型中的任务主链：数据助理对话与 Dashboard 同屏，右侧直接出现由当前任务关联的可编辑看板。用户无需离开任务寻找结果，也可通过分隔线调整左右区域，或切换全屏看板。默认状态下，每个组件右上角都有独立编辑图标，用户不必先切换整个页面的编辑模式，就能准确进入当前组件的设置。

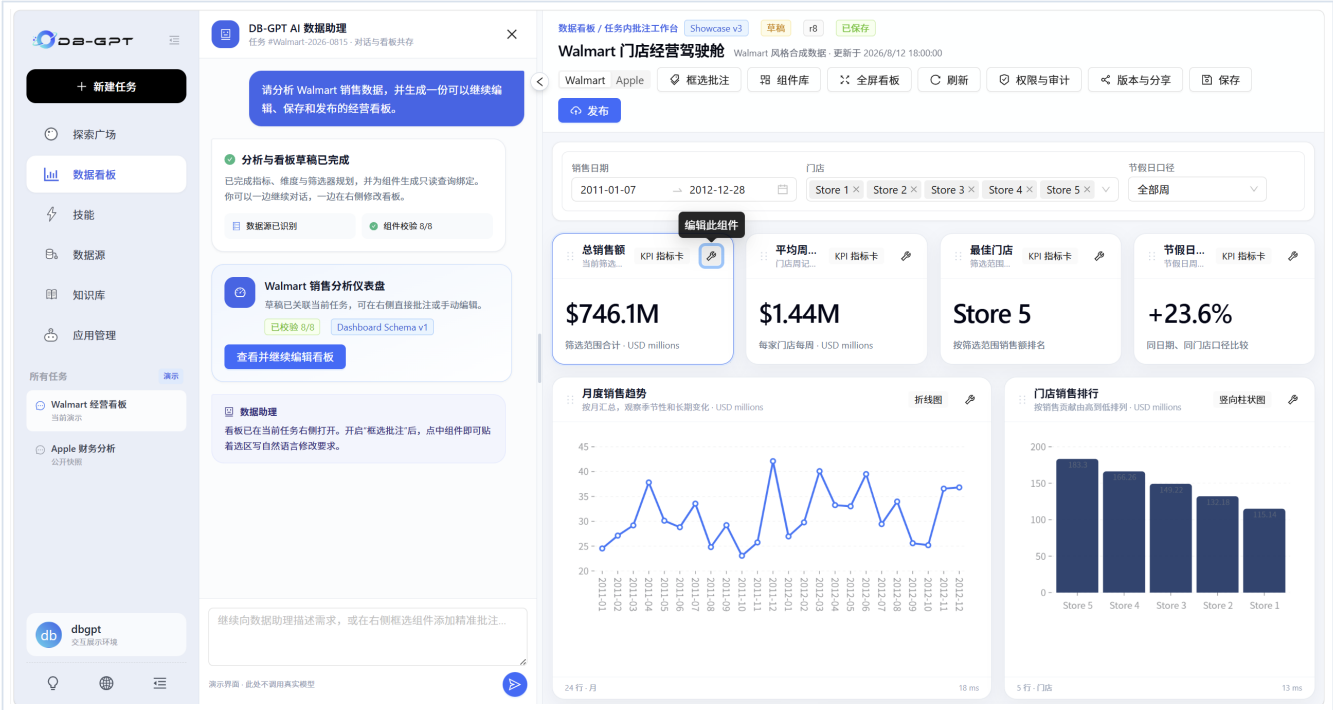


图 1 任务内对话与 Dashboard 同屏，每个组件提供独立手工编辑入口

这一界面验证了四个关键产品判断：任务页是生成和继续编辑的第一入口；对话与看板需要共享任务上下文；组件必须既能精确手工编辑，也能被 Agent 定位修改；资产库只管理已保存结果，嵌入式与独立页面共用相同 Dashboard ID、Schema 与修订号。

说明：该公开页面是交互原型，演示数据随静态站打包，当前不会调用真实模型或生产数据库。正式项目将把界面接入 DB-GPT 已有 ReAct Agent、SSE 和数据源执行链路。

七、框选批注与手工编辑

框选批注用于解决“用户知道哪张图不对，却难以用长文字描述位置”的问题。开启批注后，组件右上角的操作会切换为批注入口。用户点击组件、图表数据点或表格单元格后，系统在原位附近打开评论框，并自动携带目标对象、筛选条件、组件配置和当前修订；用户只需写“改成横向条形图并按销售额降序”等自然语言要求，Agent 就能知道“改哪一块、基于什么上下文改”。

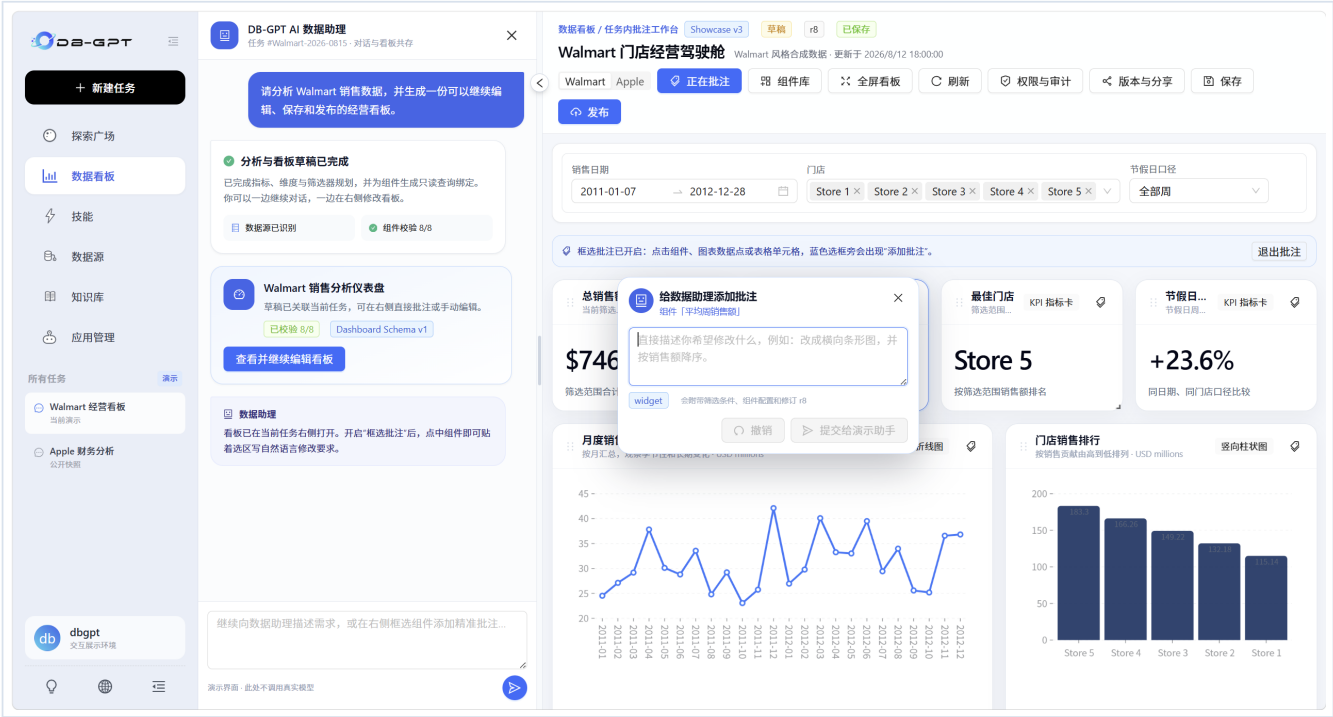


图 2 任务内框选具体组件后，就地填写面向数据助理的自然语言修改意见

正式实现的变更流程固定为：

选中目标 → 输入评论 → Agent 生成变更建议
→ 显示前后差异 → 用户应用或放弃
→ 服务端重新校验 → 保存为新修订

同时保留手工编辑入口，避免用户必须依赖模型。两种入口互不混淆：未开启批注时，组件图标打开手工编辑抽屉；开启批注时，点击目标打开自然语言评论框。标题、说明、图表类型、字段映射、参数化 SQL、样式、排序和筛选器都可由属性面板修改。AI 编辑与手工编辑操作最后都落到同一 Schema 和同一修订历史。

八、Schema 驱动编辑器

8.1 组件库与扩展方式

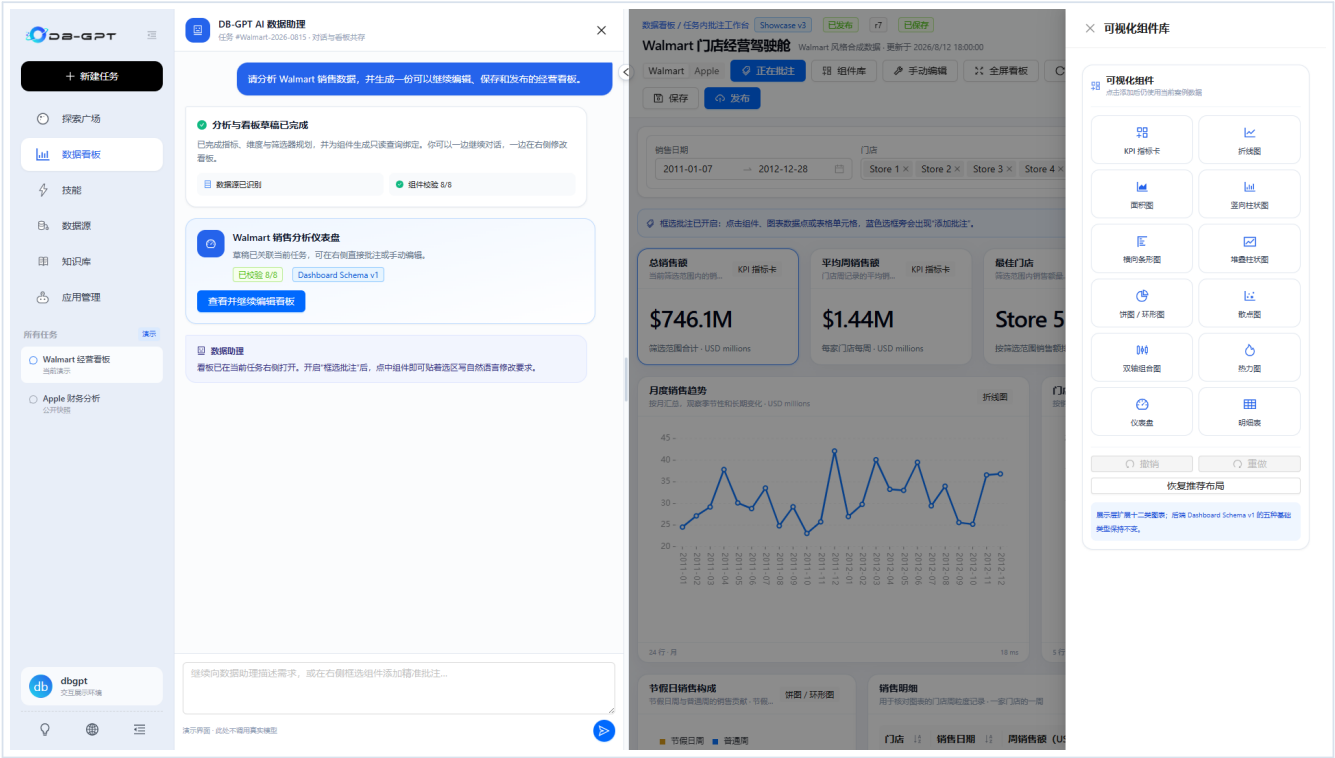


图 3 组件库：核心类型与展示层扩展类型共用同一数据合同

正式项目优先完成官方要求所需的五类核心组件和三类筛选器；额外视觉类型通过可选 `style.visualization` 或后续 Schema 小版本扩展，避免为了图表数量破坏稳定合同。新增组件默认处于“尚未配置查询”状态，引导用户依次完成 SQL、输出字段、图表映射与试运行，不伪造示例结果。

8.2 手工设置与查询可解释性

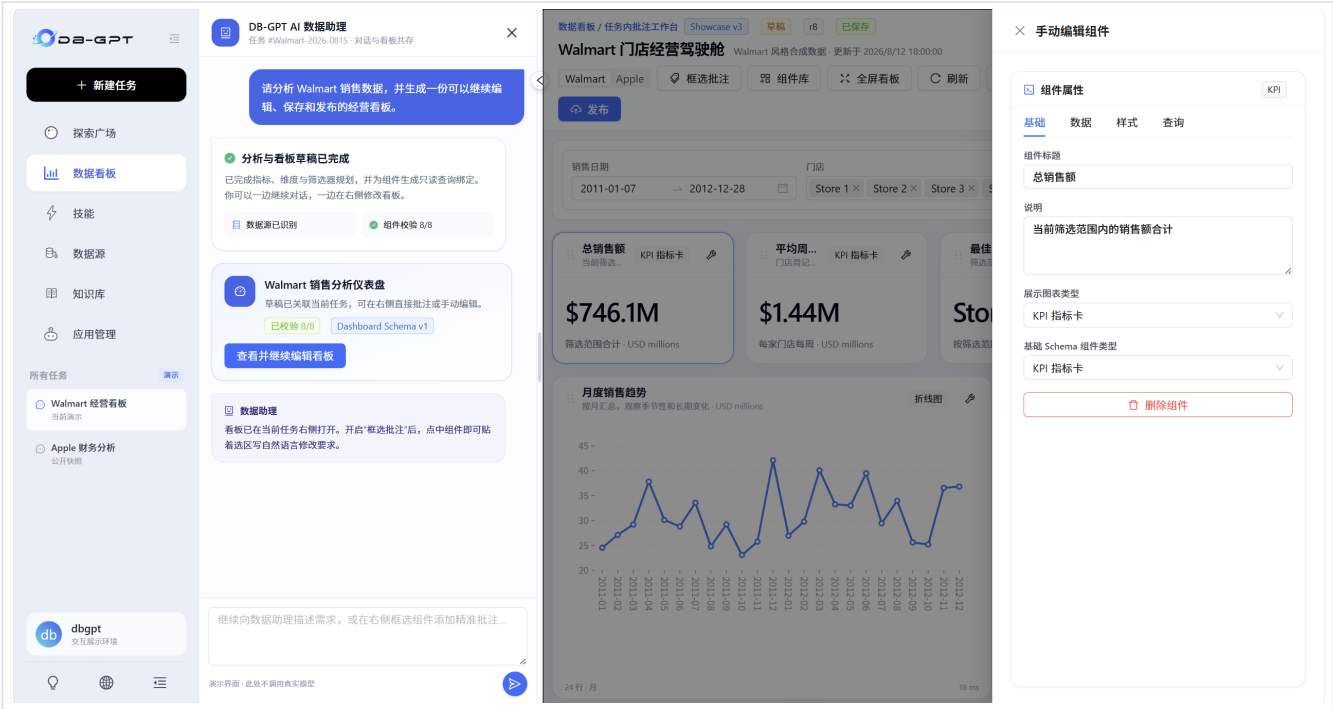


图 4 点击组件右上角编辑图标，右侧抽屉只编辑当前组件的基础、数据、样式与查询配置

手工抽屉始终绑定当前选中的组件，分为基础、数据、样式和查询四部分。用户可以修改标题与说明、切换展示类型、调整字段映射与格式，并查看或试运行参数化 SQL；删除操作也只作用于当前组件。编辑器同时区分加载、空数据、查询失败、字段映射错误、整体失败和修订冲突。单组件失败只在对应卡片显示错误，其他组件继续展示最近一次成功结果。

九、后端、持久化与 API

9.1 数据表与修订

- `dbgpt_dashboard`: 当前草稿、所有者、会话 ID、来源回合、资产状态、当前修订号、状态和更新时间;
- `dbgpt_dashboard_revision`: 不可变发布 Schema、数据快照、校验摘要、分享令牌哈希和发布时间。

保存采用乐观并发: 前端提交 `expected_revision`; 如果服务器修订号已经变化, 返回 409, 让用户重新加载或另存, 不能静默覆盖他人修改。

9.2 核心接口

方法	路径	作用
POST	<code>/api/v1/dashboards</code>	创建 generated 草稿
GET	<code>/api/v1/dashboards/{id}</code>	读取草稿及当前修订
PUT	<code>/api/v1/dashboards/{id}</code>	保存并在首次保存时资产化
POST	<code>/api/v1/dashboards/{id}/validate</code>	校验 Schema 与全部查询
POST	<code>/api/v1/dashboards/{id}/widgets/{widget_id}/preview</code>	试运行单组件
POST	<code>/api/v1/dashboards/{id}/refresh</code>	按筛选器刷新可执行组件
POST	<code>/api/v1/dashboards/{id}/publish</code>	生成不可变发布修订
GET	<code>/api/v1/dashboards</code>	按会话、来源、资产状态和状态筛选
GET	<code>/api/v1/public/dashboards/{token}</code>	匿名读取发布快照

9.3 旧功能兼容

现有 `chat_dashboard` 不直接删除。适配器将旧 `ReportData.charts` 导入为新草稿, 不原地修改旧记录; 功能开关与独立路由保留回退能力。正式合并前会先跑上游基线测试, 再区分“上游已有问题”和“本项目引入问题”。

十、SQL 与分享安全

SQL 安全不能只靠关键词正则。计划采用多层防线：

- 1 使用项目已有 `sqlglot` 解析 SQL 抽象语法树；
- 2 只允许单条只读 `SELECT` 或 `CTE`；
- 3 禁止 `DDL`、`DML`、多语句、`PRAGMA`、文件读写、命令执行和危险扩展函数；
- 4 用户筛选值只使用命名参数绑定，禁止字符串拼接；
- 5 表名、字段名和排序方向采用服务端允许列表；
- 6 使用调用者现有数据源权限，不自动提升权限；
- 7 设置查询超时、组件最大行数和全看板有界并发；
- 8 服务端再次截断结果，避免只信任数据库驱动；
- 9 预览、刷新和发布都重新校验权限与数据源边界。

发布链接只读取发布时固化的 Schema 与允许分享的数据快照。匿名用户不能通过 token 重新执行 SQL；数据库只保存 token 哈希。分享页可以在冻结数据切片上改变日期、门店等筛选器，所有计算在浏览器本地完成，不回写默认值，也不影响其他查看者。

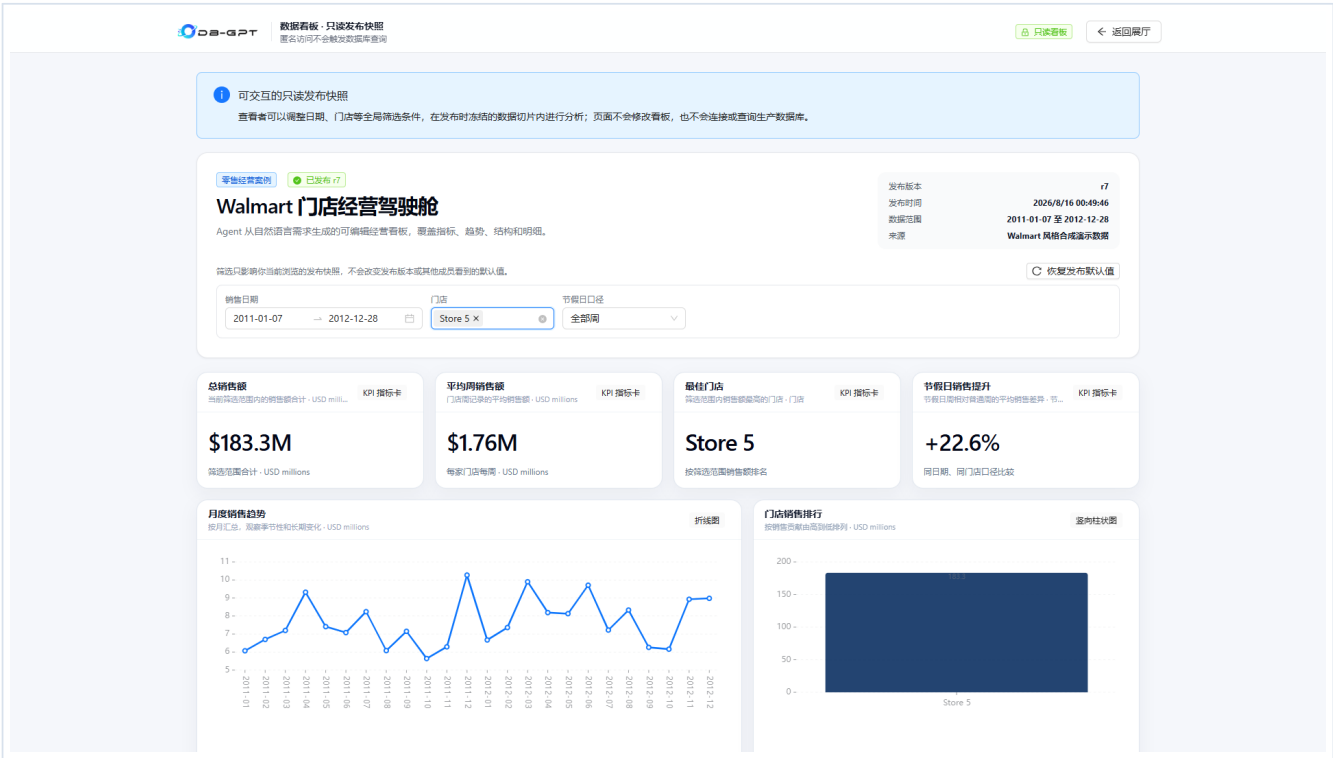


图 5 分享查看者仅筛选 Store 5 后，KPI 和图表基于冻结数据切片联动变化

公开原型中全部门店为 **\$746.1M**，仅 Store 5 为 **\$183.3M**；两次筛选变化期间无网络请求。这证明“可交互分享”与“匿名实时查库”可以分离。

十一、草稿、发布、版本与分享

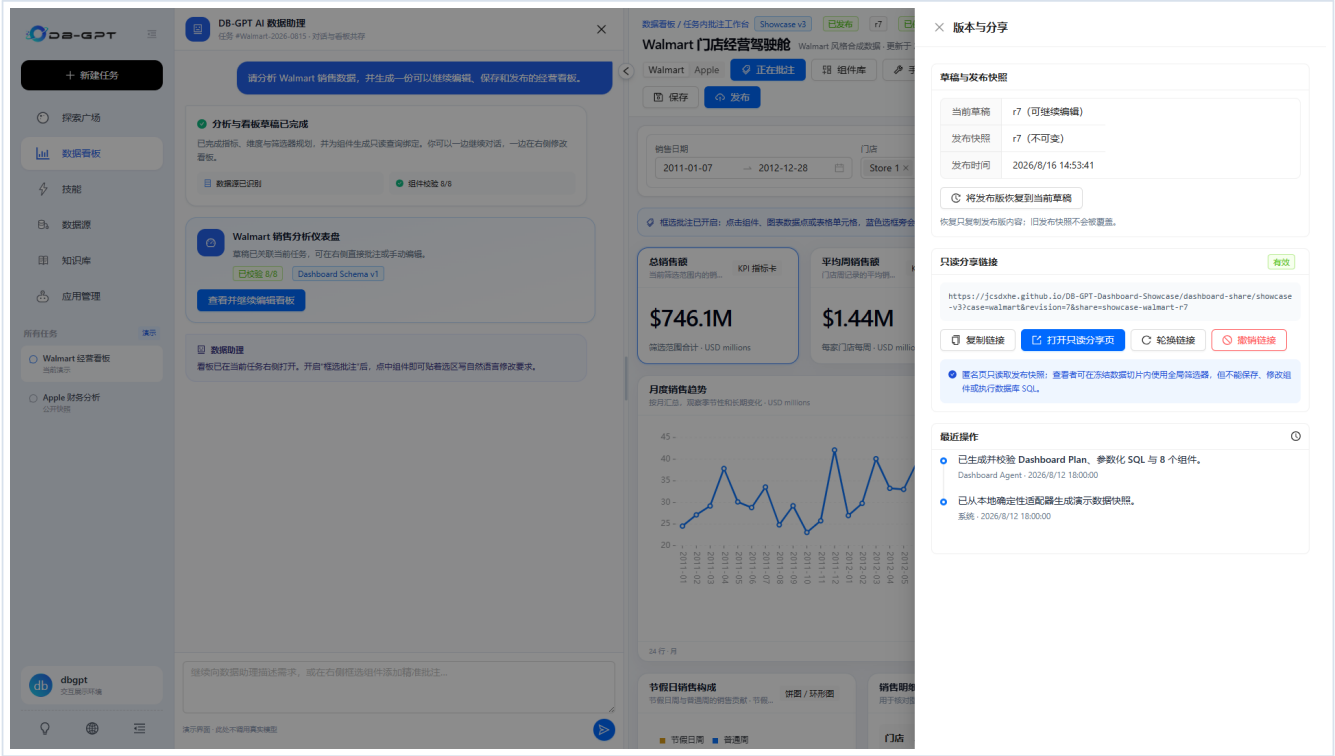


图 6 编辑草稿与不可变发布版本分离，支持复制、打开、轮换、撤销和恢复

生命周期设计为：

generated（任务生成、可恢复但不进入资产库）

→ saved（用户首次明确保存、进入资产库）

→ published（生成不可变发布修订和分享链接）

后续编辑只改变草稿，不会覆盖已经发布的快照。需要把旧发布内容继续修改时，应“恢复为新草稿修订”，而不是篡改历史版本。分享令牌可以轮换或撤销；正式实现还将支持过期时间。发布失败不能影响已保存草稿，单组件失败不能让整个 Dashboard 消失。

公开演示中的权限与审计页面只是对预研能力的静态交互说明，不代表 GitHub Pages 提供真实企业认证。正式 v1 只需正确使用 DB-GPT 当前身份和数据源权限边界，不把企业级权限平台重构列为阻塞交付。

十二、两个通用演示案例

12.1 Walmart 门店销售

Walmart 案例使用确定性合成数据，并明确不代表真实 Walmart 经营结论。它验证：总销售额、平均周销售额、月度趋势、门店排行、节假日对比、日期范围、门店多选、节假日筛选和销售明细可以在同一 Schema、查询与刷新链路中工作。

12.2 Apple 财务分析

Apple 案例使用 FY2022-FY2024 Form 10-K 中少量公开结构化事实，保存 SEC 来源 URL、提取时间、金额单位和口径。它验证：年度收入、利润、利润率、资产、产品收入结构、资金来源结构和财务明细能够复用相同编辑器与发布服务。

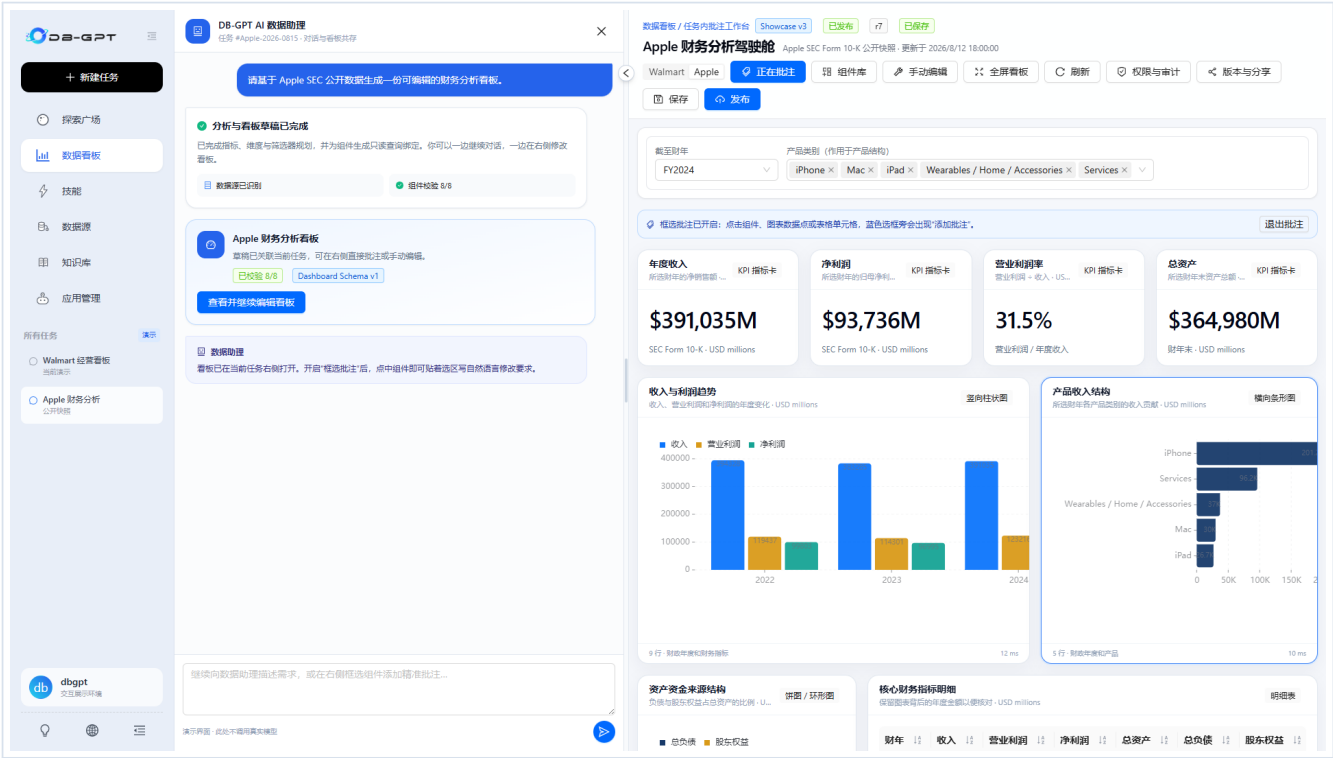


图 7 Apple 财务案例与 Walmart 共用同一任务主链和渲染器

两个案例只更换数据与 Schema，不复制两套页面。零售与财务主题都能工作，才能证明模块是通用能力而不是两张定制静态图。

十三、测试、质量门与当前证据

13.1 正式项目测试层次

- **后端单元/接口测试**：Schema、DAO、权限、修订冲突、SQL AST、参数绑定、超时、局部失败、发布不可变、分享安全、旧版导入；
- **前端测试**：五类组件、三类筛选器、增删切换、SQL 面板、保存冲突、部分失败、嵌入/独立一致性；
- **数据库测试**：SQLite 与 MySQL 的建表、CRUD、刷新、发布、快照大字段和参数筛选；
- **E2E**：自然语言生成、任务内打开、编辑、刷新、保存重开、资产库定位、返回来源任务、发布与无登录分享；
- **真实模型评估**：明确/模糊需求、不存在字段、SQL 首次失败、空数据、中英文混合、继续修改已有看板等固定提示集。

13.2 量化验收目标

指标	目标
确定性 Schema 合法率	100%
危险写 SQL 执行次数	0
固定提示修复后 Schema 合法率	≥ 95%
组件 SQL 修复后成功率	≥ 90%
保存、重开、刷新、发布 E2E	全部通过
单组件失败时成功组件保留	100%
匿名分享页数据库查询	0 次

13.3 已验证证据与诚实边界

范围	当前证据	边界说明
完整后端预研原型	103 passed; 真实 MySQL 8.4.10 CI 通过	私有预研分支，不等于正式社区合并结果
完整前端预研原型	9 文件、15 项 Vitest; Dashboard TS 0 错误	上游全仓历史 TS 错误另行记录，不归零包装
公开 Showcase v3	17 文件、54 项测试; 6 条 Playwright; 9 个静态页面	静态数据与本地交互，不调用真实模型/生产数据库
真实模型稳定率	待在正式配置和真实数据源上评估	不用模拟结果冒充真实模型效果

十四、六周开发与交付计划

开源之夏项目周期按中选后两个月准备，但我将核心工作按六周倒排：**第 1-4 周完成全部功能主链，第 5 周完成真实环境测试、问题修复、文档、宣传视频 demo 和交付包，第 6 周只做提交前审查与必要修正，不再加入新的主要功能。** 每周均包含 Coding、Testing、Docs，并在第 1-5 周内部预留约 15% 弹性处理上游变化、模型波动和 Debug；官方周期内剩余时间作为外部阻塞和社区反馈的兜底，不作为核心功能按时完成的前提。

周次	阶段目标	Coding	Testing / Docs 与验收产出
第 1 周	需求对齐、Schema 冻结与后端骨架	阅读数据助理主链；定义 Pydantic、JSON Schema、TypeScript；建立实体、DAO、迁移和任务关联字段	合法/非法/迁移样例；设计评审；Schema、CRUD 与修订冲突测试全绿
第 2 周	确定性后端与基础渲染闭环	完成参数化 SQL 安全层、预览/刷新、乐观并发、五类组件、三类筛选器和错误状态	SQLite/MySQL 建表与接口测试；无模型完成创建、保存、刷新、发布；固定 Schema 完整渲染
第 3 周	任务主链、共享编辑器与 Agent 接入	完成嵌入式/独立共享核心、组件手工抽屉、布局与查询面板；接入 DashboardPlan、QueryDraft、SSE 和局部修复	历史任务恢复；保存重开不丢失；单组件失败保留；模拟模型输出不能绕过校验
第 4 周	批注修改、发布分享与两个案例	完成组件/数据点结构化批注、受限 ChangeSet、不可变快照、可筛选匿名页、Walmart、Apple 和旧版适配；打通自然语言到分享页	AI/手工修改落入同一 Schema；两案例完整 E2E；来源、指标口径、分享安全和兼容文档完成
第 5 周	全量测试、视频与正式交付包	接入真实模型和真实数据源；修复 Schema/SQL 失败；收口错误恢复、性能边界和关键体验问题；拆分清晰提交	十类真实提示评估；后端、SQLite/MySQL、前端、构建、Playwright 和人工浏览器全量回归；用户/开发/API 文档、测试报告、已录制剪辑的宣传视频 demo、演示脚本和 PR/MR 交付包全部完成
第 6 周	提交前审查	不新增主要功能；根据导师审查意见只修复阻塞提交的问题，并同步必要的 upstream 变更	逐项复核官方要求；代码、安全、密钥、许可证、数据来源、文档一致性和可复现性审查；最终回归、演示彩排、材料校对并提交

14.1 每周节奏

- 周初：确认本周验收条件和接口变更；第 6 周改为确认审查清单，不再扩张功能；
- 周中：提交小而单一的实现与测试，及时同步阻塞；
- 周末：运行全量质量门，更新文档、测试证据与风险清单；
- 每周向导师同步 1-2 次：已完成内容、可复现证据、下周计划和需决策问题；
- 第 1-5 周的 15% 弹性时间不提前安排功能，只用于模型/环境不稳定、上游冲突和缺陷修复；第 6 周完整保留给提交前审查。

十五、风险、回滚与范围控制

风险	影响	应对与回滚
模型输出不稳定	Schema/SQL 不合法	两阶段生成、严格校验、逐组件试运行、局部失败保留、模拟模型回归
SQL 安全与越权	数据泄露或写操作	AST 只读、绑定参数、允许列表、最小权限、超时/行限；默认拒绝
任务页与资产库分叉	两套数据难维护	共享 Schema/编辑器/API；用同一 ID 与修订做一致性测试
前端范围过大	延误核心链路	优先五类组件和三类筛选器；扩展图表列为可选增强
与上游冲突	合并困难	固定基线、定期同步 <code>upstream/main</code> 、小提交、功能开关与旧版适配
数据口径错误	看板可信度下降	强制保存单位、粒度、来源、新鲜度；发布前校验摘要
匿名分享风险	绕过数据库权限	只读不可变快照；筛选只在允许分享的数据切片中本地计算
学期课程与项目并行	测试和文档时间被挤占	当前每周最多七次课，结项前无考试；固定投入约 30 小时并保留 15% 缓冲

P0/P1 问题（数据错误、SQL 安全、保存丢失、发布泄露、主链不可用、两个案例失败）不得延期。只有不影响核心验收的体验增强可进入 `KNOWN_LIMITATIONS.md`，并记录复现、影响、临时方案和推荐修复。

十六、项目交付物与官方要求对照

官方要求 1

基于当前的数据助理设计并实现 Dashboard schema 完整的模块代码以及设计方案文档。

计划交付：Pydantic 模型、JSON Schema、TypeScript 类型、合法/非法/迁移样例、Schema 校验测试、架构设计、字段说明、Agent 规划协议、SQL 安全设计、API 文档、任务关联与旧版适配说明。

官方要求 2

完成看板编辑发布及两个演示案例以及对应的文档或者宣传视频 demo。

计划交付：任务内嵌编辑器、组件级手工编辑与结构化批注入口、资产库入口、五类核心组件、三类筛选器、布局编辑、SQL 试运行、保存刷新、不可变发布、可筛选只读分享、Walmart 与 Apple 案例、自动化测试、用户/开发/API 文档、测试报告、演示脚本和完成录制剪辑的宣传视频 demo。上述实现、测试、视频和交付材料均在第 5 周结束前完成，第 6 周仅作提交前审查与必要修正。

十七、个人基础与投入说明

17.1 教育背景与时间可行性

我就读于天津工业大学软件工程专业，目前大三。现阶段课程表每周最多七次课，周一和周末没有固定课程，大部分晚间及无课时段均可用于开发；按当前教学安排，项目结项前没有考试。计划工作日累计投入约 14-16 小时，周末累计投入约 14-16 小时，稳定保证每周约 30 小时，并将其中约 15% 留作 Debug、上游冲突和模型/环境波动缓冲。

我曾获蓝桥杯全国软件和信息技术专业人才大赛 C/C++ 程序设计大学组省级三等奖。该经历帮助我形成了拆解问题、定位错误和持续验证的习惯。

17.2 技术与项目实践

我掌握 HTML、CSS、JavaScript、TypeScript、React、Python 和 SQL 的基础知识，完成过以下个人项目：

- [AI 问答及学习督导平台](#)：围绕学习问答、任务督导和页面交互进行完整项目实践；
- [电影推荐系统](#)：围绕推荐结果展示、数据处理和用户交互完成项目实践。

这些经历使我具备把需求拆成页面、接口和数据处理流程的基本工程能力。BI 方面，我了解连接数据源、理解表结构、SQL 查询、指标计算、筛选与图表展示的基本流程，也实际运行过 DB-GPT 的数据库分析、销售分析和财报分析示例。

AI 方面，我长期使用主流大语言模型和 AI Agent，了解不同模型在编程、推理、长文本、工具调用和成本方面的差异，实践过 API 接入、提示词与上下文、Skill/工具调用、RAG 与 LangChain 基本流程，也使用过跨设备方式远程让 AI Agent 在电脑上执行任务。这些体验让我重视准确性、任务连续性、错误恢复和用户操作成本。

17.3 开源准备与承诺

目前我尚未向 DB-GPT 官方仓库提交正式 Issue 或 PR；申请阶段完成的是基于官方 `main` 的环境搭建、源码阅读、独立分支预研、测试记录和公开交互原型。中选后，我会先与导师确认正式边界，再按社区规范整理提交，确保每项进入申请书和代码审查的内容都能够由我自己解释、维护和持续迭代。

十八、参考资料

- 1 DB-GPT 官方仓库: <https://github.com/eosphoros-ai/DB-GPT>
- 2 开源之夏项目详情: <https://summer.ospp.ac.cn/org/prodetail/26f420021>
- 3 公开交互原型（任务内编辑与批注）: <https://jcsdxhe.github.io/DB-GPT-Dashboard-Showcase/dashboards/showcase-v3/?case=walmart&mode=editing>
- 4 公开发布快照（可筛选只读视图）: <https://jcsdxhe.github.io/DB-GPT-Dashboard-Showcase/dashboards/showcase-v3/?case=walmart&mode=published>
- 5 JSON Schema 2020-12: <https://json-schema.org/draft/2020-12>
- 6 SQLGlot: <https://github.com/tobymao/sqlglot>
- 7 OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- 8 ReAct 论文: <https://arxiv.org/abs/2210.03629>
- 9 React-Grid-Layout: <https://github.com/react-grid-layout/react-grid-layout>
- 10 Apache Superset: <https://github.com/apache/superset>
- 11 SEC EDGAR: <https://www.sec.gov/edgar>

申请承诺： 我会遵守开源之夏与 DB-GPT 社区规范。现有私有原型和公开静态演示只用于申请阶段的技术验证，不提前向官方仓库提交，也不冒充正式活动成果。若中选，我将在导师确认的范围和开发周期内独立完成正式实现、测试、文档和 PR/MR，并对自己提交的每项内容负责。